



A formally verified blockchain-based decentralised authentication scheme for the internet of things

Khizar Hameed¹ · Saurabh Garg¹ · Muhammad Bilal Amin¹ · Byeong Kang¹

Accepted: 22 April 2021 / Published online: 15 May 2021

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract

With the proliferation of intelligent devices in the Internet of Things (IoT), the security of IoT devices is becoming a major concern as an attacker can exploit network services via network capture or deployment of malicious devices. Therefore, IoT device authentication is a key security requirement. Conventional authentication approaches have proved insufficient in dealing with a large number of IoT devices because of various shortcomings, such as centrally designed architectures and expensive cryptographic primitives. Blockchain technology has led to the creation of decentralised IoT authentication solutions with their desirable characteristics. Due to the integration of IoT and Blockchain technology, however, performance is one of the significant challenges. Existing Blockchain-based authentication schemes create extra storage, computing and communication overheads on the Blockchain which restricts its use for different IoT networks. In this paper, we design an efficient decentralised Blockchain-based authentication scheme aimed at achieving a secure authentication for IoT devices using the token mechanism. We design the Ethereum Blockchain network model to implement our scheme, in which smart contracts provide secure connectivity between miner nodes and IoT devices. We demonstrate our proposed scheme with an IoT-based smart home case study, which shows significant performance in computation and communication overheads compared with existing Blockchain-based authentication schemes. The modelling, analysis and verification of the proposed scheme, using high level Petri nets (HLPNs) and Z3 SMT solver are also provided to prove correctness and to verify the claims of our model.

Keywords Internet of things · Blockchain · IoT Device · Authentication · Security · Formal modelling · Verification

✉ Khizar Hameed
hameed.khizar@utas.edu.au

Extended author information available on the last page of the article

1 Introduction

Today's world is becoming digital by integrating various intelligent physical objects that are connected and working together to build a network known as the Internet of Things (IoT) [1]. IoT is a suite of different types of devices, ranging from personal computers to autonomous smart objects equipped with sensors, actuators and a variety of software capabilities [2]. This technology can make conventional Internet-based work models smart and automated with a wide range of applications, such as smart homes, smart cities, smart vehicles, smart health services [3] and smart industries [4–6].

The main objective of these collaborative and intelligent applications is twofold: (i) perform the desired features in the underlying field and (ii) provide secure communication between interactive objects. With the massive growth and accessibility of smart devices on the IoT network, attackers have been able to deploy malicious devices and perform fraudulent operations in order to gain full control over the network [7]. Therefore, the most critical challenge for IoT networks is to protect IoT devices from adversaries sending malicious data [8, 9].

Authentication is a key requirement for the security of IoT devices, including the identification of devices, users and applications across multiple organisations. Authentication is an essential step that can restrict unauthorised user access to the deployment of invalid or malicious network services [10]. Most authentication approaches are specifically designed for high performance computing infrastructures and are, therefore, not suitable for resource-intensive IoT devices due to their limited power, storage and battery capacity [11]. Thus, lightweight authentication solutions need to be designed and implemented for the security of IoT devices [12, 13].

Another major problem identified in existing authentication schemes is the use of single server and centralised cloud authorities for network-wide registration, surveillance and authentication of IoT devices [14]. However, most centralised systems have a single point of failure problem in which an attacker can compromise the entire system by capturing or installing duplicate devices on the network in order to perform malicious activities. This action enables the attacker to take control of the IoT network, with the main aim of capturing the network information and the personal data of the users [15, 16].

Blockchain technology has received substantial attention in recent years from both academics and research industries in which it can provide a secure way to address problems such as distributed denial-of-service (DDoS), misuse of data and content manipulation in existing centralised security schemes [17–19]. Blockchain is a growing list of secure records, famous for their main characteristics such as decentralisation, distributed tamper-proof ledger, transparency and auditability [20]. Based on the characteristics of Blockchain technology, many investigators have expanded and replaced existing IoT architectures, which required trusted parties to regulate and validate records [21, 22].

The Blockchain-based decentralised authentication system facilitates peer-to-peer (P2P) security for IoT devices, instead of depending on the central server that may lead to network interruption and failure issues [23]. In addition, the

centralised server may also sometimes involve the manipulation of stored data; this has had an impact on the accountability of the network. However, in a few cases, centralised servers can perform better and make various tasks easier, as decentralised and hierarchical network management is not a trivial task. Blockchain-based IoT security systems replace existing centralised IoT security models that allow future connected devices to interact and share data directly with each other, in a transparent and tamper-proof manner [24, 25]. In Blockchain, trust is established and managed through a variety of consensus algorithms between distribution parties [26, 27].

Although the integration of Blockchain technology into IoT has numerous advantages, such as decentralisation, transparency, security, autonomy and immutability, it also faces numerous challenges in terms of performance scalability and data storage [28]. For example, few Blockchain-based IoT systems have been proposed with the aim of authenticating IoT devices for secure communication through the network but the proposed schemes are creating additional overheads in terms of system storage, communication and computing [17, 29, 30]. For example, [31] proposed a Blockchain-based IoT device authentication scheme for secure communication; however, the scheme creates additional system storage, communication and computing overheads due to the use of multiple and repeated device attributes. In another scheme [29], which is designed to create and store digital identities for IoT devices on a distributed ledger, the identity creation mechanism introduces processing overheads in the system.

In this paper, we propose an efficient Blockchain authentication scheme for IoT devices to resolve shortcomings in existing IoT authentication systems. We have used Ethereum Blockchain as the primary Blockchain network that enables smart contracts to ensure secure communication between IoT devices. To authenticate and validate IoT devices on the network, we use a token mechanism that contains unique credentials for each device, such as device IDs, addresses, public and private keys. Our experimental findings show that the proposed scheme has improved functionality, performance and efficiency for large IoT networks.

To design the Blockchain-based efficient authentication scheme for IoT devices, the following are the contributions to this paper.

- We highlight the issue of device authentication in IoT scenarios and then explore the significance of integrating Blockchain technology into IoT-based security solutions.
- We develop an Ethereum Blockchain-based token authentication mechanism for IoT devices.
- We design smart-contract-based efficient device authentication algorithms for secure connectivity between miner nodes and IoT devices.
- We provide a comparison of the proposed authentication scheme with state-of-the-art Blockchain-based IoT authentication schemes in terms of computation overhead, communication overhead and financial cost.
- We also provide formal modelling, analysis and verification using high level Petri nets (HLPNs) and Z3 solver to verify the claims of our proposed authentication model.

The rest of the paper is structured accordingly: The background and related work of existing systems and their shortcomings are discussed in Sect. 2. We provide a detailed description of our proposed methodology with the network and attacker models, proposed algorithms and execution flow in Sect. 3. The formal modeling, analysis and verification of our proposed methodology is described in Sect. 4. In Sects. 5 and 6, the evaluation framework and performance evaluation are then described accordingly. Finally, we summarise our work in Sect. 7 and discuss future research work.

2 Background and related work

This section describes Blockchain technology and Ethereum Blockchain as a preliminary or background to our work and then provides an overview of the existing Blockchain-based IoT authentication schemes in the related work subsection.

2.1 Preliminaries

2.1.1 Blockchain

The basic concept of Blockchain technology was to ensure the processing of payments between two parties without the need for a trusted third party and to maintain a tamper-proof ledger for secure transactions. The first “Bitcoin” cryptocurrency Blockchain application to become popular was originally launched by Satoshi Nakamoto in 2008 [18]. Blockchain technology incorporates P2P networking, in which individual nodes called “miner nodes” accept and verify transactions before being added to the Blockchain. Each block can contain a number of transactions that are structured in the Merkle hash tree. In addition to transactions, time stamps, previous block hash value and block version information are also part of each block structure. The previous block hash value is stored in the next consecutive block to create an entire Blockchain in order to ensure that the stored data remains unchanged [19].

2.1.2 Ethereum

Ethereum Blockchain is the most popular, open and public platform for the development of decentralised applications. The design of the Ethereum Blockchain consists of thousands of virtual machines, formally known as the Ethereum Virtual Machine (EVMs) [25]. The EVM is responsible for the execution of functionalities written in smart contracts to satisfy all the tasks required. In addition, since the introduction of Bitcoin, Ethereum has been used to transfer funds without requiring third parties. Unlike Bitcoin, Ethereum used its cryptocurrency known as Ether to consolidate funds across numerous applications [27].

2.2 Related work

This section discusses the integration of Blockchain technology into the IoT context in terms of self-configuration and self-management, as well as the challenges and limitations associated with current Blockchain authentication schemes for IoT devices. Finally, this section analyses the different security issues associated with Blockchain systems and the recently suggested solution for addressing these concerns at various levels.

IoT networks are made up of a large number of heterogeneous mobile devices that are connected to the internet, promising a myriad of opportunities for a massive selection of distributed applications and services. However, the IoT fundamentally alters the internet paradigm, owing to the millions or even billions of embedded devices that would almost certainly be unable to be handled centrally by cloud providers due to their lack of scalability. Therefore, in such situations, when cloud providers are unable to meet the requests of all IoT devices, these IoT devices must self-manage their software/hardware configuration and resource usage. One such mechanism. Self-configuration and self-management systems can be achieved by the use of measurement-based learning and adaptation to changing system environment and application requirements. However, current methods have some trust-related drawbacks and therefore extensive research on self-optimisation and self-configuration issues is needed [32].

One major limitation of current authentication systems for IoT is that they are predominantly centralised-controlled, resulting in device hacking, compromised authentication mechanism and information sharing issues. Since authentication of IoT devices is vital in registering IoT devices before entering the network to perform various activities. As a result, authentication is a crucial challenge for securing IoT devices, requiring substantial effort to propose efficient solutions. Based on multiple mechanisms, various methods have been proposed for authenticating IoT devices in various settings, ranging from home security to large-scale industrial setups. The following is a description of the various proposed authentication mechanisms based on Blockchain technology [33].

For example, Huh et al. [34] suggested a Blockchain-based IoT framework for monitoring and maintaining a wide range of network devices. Smart contracts are written and deployed on the Ethereum Blockchain for key management and fine-grained access policies. However, the RSA (Rivest–Shamir–Adleman) cryptosystem used to generate public and private keys to handle the various operations of devices has been proven costly for resource-constrained IoT devices. Authentication and authorisation of IoT systems using the web trust models are difficult tasks since various cases are managed collaboratively and interactively. Durand et al. [35] thus suggest the concept of a Blockchain-based public key infrastructure (PKI) interface for a restricted network environment, in order to achieve scalability, security and openness. However, the scheme only emphasises the theoretic scenario of the proposal and provides no implementation in real-time IoT scenarios so as to test its applicability.

In a further study, Li et al. [36] proposed the decentralised authentication Blockchain framework for IoT devices to eliminate data tempering and data spoofing

vulnerabilities in current centralised certification schemes. The secure and tamper-proof feature of Blockchain technology makes it possible to keep a device record securely in a distributed ledger. In the proposed framework, the users are provided with a unique identity against a single IoT device which is saved in the distributed ledger in order to authenticate without a central authentication authority. However, this proposed method focused on the theoretical analysis in the proposed framework and did not provide the evaluation results to assess the applicability of performance in terms of computation and communication.

Almadhoun et al. [37] have put forward a Blockchain-based user authentication system that allows fog nodes to communicate with Ethereum Blockchain through smart contracts. The proposed system verifies by providing the user with proper access and control of the administrative tasks, such as addition or deletion of devices to the IoT ecosystem. Nevertheless, the proposed system is restricted to the extent that it was only developed and tested with the Remix IDE and did not provide real-time computing and communication analysis.

Fayad et al. [30] proposed a Blockchain-based IoT gateway authentication and authorisation scheme to assign core responsibility to the gateway for the authentication of IoT devices in different cases. The proposed scheme can verify the mobility of IoT devices via a Blockchain gateway. Lau et al. [17] suggested the authenticated application configuration protocol (ADCP) to store structured data and files on the Blockchain, in order to manage the security problem of a large number of IoT based devices. In this proposed method, a single ID number is assigned for the verification process in which the Blockchain provides an immutable database to store the IDs. The suggested scheme, however, creates increased overhead computation and storage needs in order to move the data of a device into a Blockchain.

Hammi et al. [31] proposed the Bubble of Trust, a decentralised Blockchain authentication system for authenticating and recognising IoT devices. In this proposed scheme, device registration and secure communication are the responsibility of each secure area called the “Bubble”. The limitation of the proposed mechanism is that it only considers the device association cost with the Blockchain and does not focus on the registration process for the IoT devices, thereby limiting its applicability for the larger IoT networks. Lee [38] proposed a new Blockchain-based identity service called BIDaaS to secure contact with telecommunication providers for mobile users. The secured connectivity between mobile users and service providers in the BIDaaS system is accomplished without prior certificates. The downside of the proposed approach is not, however, the measurement of performance which hinders the implementation of the proposed method to actual IoT situations. Lee and Kim [39] use the zero-knowledge-proof technique to protect IoT-data and address information in the Blockchain. The methodology is used on the intelligent metering system to check the details without disclosing information like a public key. The schema, however, was only checked via Remix IDE without offering scenarios in real-time and simulation. Patwary et al. [40] proposed a decentralised device-to-device authentication model based on Blockchain technology wherein the Fog devices authenticate each other at the Fog side by using Blockchain. This approach made use of the fog-based framework to authenticate and manage IoT devices. This approach, however, has limitations in terms of adding additional computation to the fog layer. Further,

Shen et al. [41] presented an efficient Blockchain-assisted secure device authentication method for cross-domain IIoT. A consortium Blockchain establishes trust between numerous areas and the authentication process using an identity-based signature. Khalid et al. [42] proposed a decentralised authentication and access control system that is applicable to a wide variety of scenarios for lightweight IoT devices. The mechanism is based on fog computing technology and the principle of a public Blockchain. However, this scheme has drawbacks in terms of performance since it only considers computation as a primary measuring criterion and does not consider storage constraints.

Apart from incorporating Blockchain technology's features into a variety of applications such as financial systems, IoT, smart grid and healthcare, Blockchain systems have been subjected to numerous outside attacks from the internet [43]. For instance, Blockchain technology is traditionally applied using conventional computers and networks and attacks on this infrastructure may compromise the Blockchain system's protection. Internet attackers can intercept the security procedure for data and transactions on the Blockchain, which frequently has to deal with a variety of malicious external and internal attacks, as well as peripheral and unintended attack vectors. As a result, the security of Blockchain technology remains critical, potentially requiring defence in terms of threat identification, prevention and effective response. One such approach is proposed in which the Blockchain security research is conducted on three levels, namely the process level, the data level and the infrastructure level, with the various levels being quantified using security policies and tools. To summarise, Blockchain protection is a significant problem that must be addressed during the system's lifecycle, from requirements specification to development and maintenance [44].

Considering the problems and drawbacks of current Blockchain-based IoT authentication and security systems, this paper aims to deliver a decentralised, efficient IoT device authentication scheme, while focusing on reducing the overhead for storage, computing and communication of devices to the Blockchain.

3 Proposed methodology

This section describes our proposed methodology to authenticate IoT devices efficiently using the network model built on Ethereum Blockchain. We also outline the assumptions for our proposed methodology in the attacker model. We have developed algorithms to secure interaction between IoT devices and miner nodes via smart contracts. This section also explains the execution flow of our proposed scheme between different components of the network model.

3.1 Network model

The network model of the proposed authentication scheme based on Ethereum Blockchain is shown in Fig. 1. In our network model, we have the following

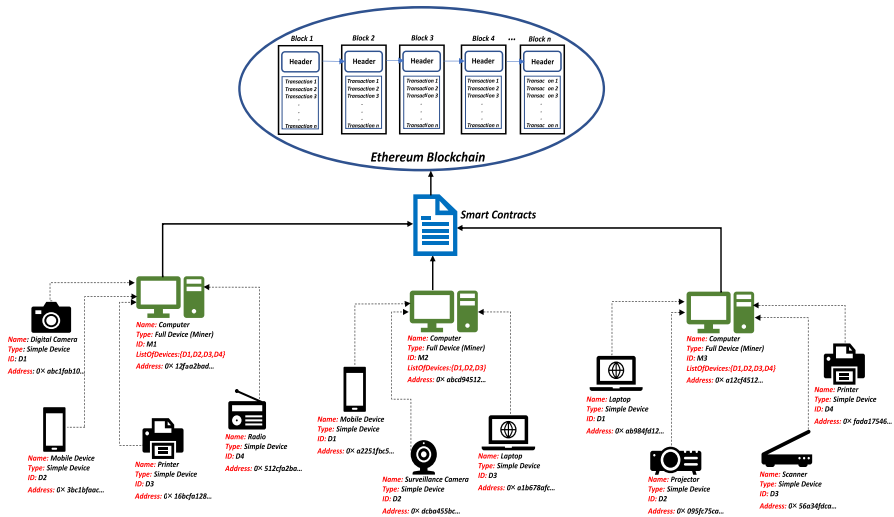


Fig. 1 Network model

communicating components: user, simple device, full device, smart contract and Ethereum Blockchain. The following are the details for each component.

3.1.1 User

A user is a person responsible for initiating the overall process and creating a smart contract on the Blockchain.

3.1.2 Simple device

A simple device is a battery-powered device with limited processing, memory and power capacity in the IoT network. In simple terms, it is a battery-powered end node device that performs a limited set of operations. Therefore, it is always controlled and managed by other sets of nodes in the IoT network (in our case, the miner node) to perform any kind of logic or function. A simple device in our architecture is a battery-powered device that interacts with the full device or a miner node for authentication, without additional storage and computers. We use the term “IoT device or node” in all the following, as an alternative to the term “simple device”. Our scheme contains a number of IoT devices with a unique IDs = $\{D_ID_1, D_ID_2, D_ID_3, \dots, D_ID_n\}$ representing device uniqueness in the network. In addition, the device addresses $D_Addr = \{D_Addr_1, D_Addr_2, D_Addr_3, \dots, D_Addr_n\}$ are in hexadecimal values which are 20 bytes long used to represent the public addresses of the IoT devices.

3.1.3 Full device

A full device in the IoT network can be a gateway to the connected devices and, overall, the network aims to verify the integrity of data before network transmission. A full device does not limit computing, storage and battery compared with a simple device and it is supposed to be available always and working with full power. A full device in our architecture is used as a mining node to manage the authentication process of simple IoT devices. In our proposed solution, the functionality of the miner nodes is twofold: (i) Grouping and managing IoT devices as clusters and (ii) Providing an interface for Ethereum network interaction. The benefit of mapping IoT devices to the mining node is that processing time and response times are reduced during decentralised validation of IoT devices. The term “miner node” is used in all subsequent sections as an alternative to “full device”.

The miner nodes also have a unique IDs = $\{M_ID_1, M_ID_2, M_ID_3, \dots, M_ID_n\}$ similar to the IDs for IoT devices. We grouped the miner nodes in a subset of nodes to group the IoT devices for authentication. Each miner node M_i can be made up of a large number of IoT devices $\{Di\}_{i=1}^n$ representing as $M_i = \{D_i, D_{i+1}, D_{i+2}, \dots, D_n\}$ where $i = 1, 2, 3, \dots, n$. Finally, miner nodes also have IoT device-like addresses, represented as M_Addr 's = $\{M_Addr_1, M_Addr_2, M_Addr_3, \dots, M_Addr_n\}$.

3.1.4 Smart contract

In our proposed solution, we used smart contracts to achieve decentralised authentication of IoT devices, initiated by miner nodes. Our smart contract includes various features, mapping of addresses, conversions and events, enabling miner nodes to manage IoT devices securely in a decentralised fashion.

3.1.5 Ethereum blockchain

Miner nodes use external calls to execute the functions written in smart contracts to communicate with Ethereum network systems, in which each function is associated with certain computing costs. In the Ethereum Blockchain, computational cost is referred to as the gas used in the smart contract for each process.

3.2 Attacker model

The objective of our proposed scheme is to ensure that IoT devices are authenticated in an efficient and decentralised manner. There are two types of nodes/devices in our network; one is a miner node that is responsible for managing and authenticating IoT devices. The other types of devices are IoT devices which must be validated before joining the network and interacting with other nodes. The attacker model for the proposed authentication scheme is shown in Fig. 2.

In our attacker model, we made the following assumptions.

Table 1 Notations

Notations	Description
M_ID	Miner ID
D_ID	Device ID
M_Addr	Miner Node Public Address
D_Addr	IoT Device Public Address
T	Token
T_{Sig}	Signed Token
$K_{M_{pr}}$	Miner Node Private Key
$K_{M_{pb}}$	Miner Node Public Key
$msg.sender$	Represents the contract owner
$Signature()$	Elliptic Curve Digital Signature function
$Hash()$	SHA256 function
$isExisting()$	Checking the existence of attributes
$isAddressExist()$	Checking the existence of Ethereum Addresses

Algorithm 1 User Registration

Input: User Credentials $\rightarrow$ Ethereum Available Addresses

Output: Successful / Unsuccessful

```

1: procedure USERREGISTRATION
2:   User.Registration()
3:   if (  $msg.sender ==$  Ethereum Addresses [0] ) then
4:     User  $\leftarrow msg.sender$ 
5:     return  $msg.sender$ 
6:     revert("User Successfully Added")
7:   else
8:     revert("User Not Added")
9:   end if
10: end procedure

```

3.3.2 Miner registration

Algorithm 2 explains how the miner nodes are added to a Blockchain. Each miner node allocates unique credentials such as the miner's id and the public address, described as M_ID and M_Addr respectively. The public address is an Ethereum generated address which is the last 20 bytes of the keccak hash of the public key. The registration process of the miner node acknowledges M_ID and M_Addr from the user and searches for the miner nodes' credentials. When Blockchain already has credentials, smart contracts reverse the transaction and give the user the error "*Miner not Added*" to inform the user of the existence of credentials.

Algorithm 2 Miner Registration

Input: Miner Credentials $\rightarrow$ M_ID , M_Addr **Output:** Miner Successfully Added / Miner Not Added

```

1: procedure MINERREGISTRATION
2:    $M\_ID \leftarrow$  Miner ID
3:    $M\_Addr \leftarrow$  Miner Address
4:   Miner_Registration()
5:   if ( $M\_ID == \text{isExisting}()$ ) AND ( $M\_Addr == \text{IsAddressExist}()$ ) then
6:     revert("Miner Not Added")
7:   else
8:     return MinerRegistration( $M\_ID$ ,  $M\_Addr$ )
9:     revert ("Miner Successfully Added")
10:  end if
11: end procedure

```

3.3.3 Token generation

The process of token generation and the integration of an IoT device is shown in Algorithm 3. To generate the token, the miner node accepted the token request which included the unique device identifier (D_ID) and the device address (D_Addr) from the node added to the Blockchain. Upon the token request, the miner node firstly verified the existence of the Blockchain credentials. If the token credentials are already stored in the Blockchain, the request for the token is denied and the transaction is returned to the user.

Algorithm 3 Token Generation

Input: Device Credentials $\rightarrow$ M_ID , D_ID , D_Addr **Output:** Token Granted/Token Not Granted

```

1: procedure TOKENGENERATION
2:   Generate Token_Request()
3:   if Token_Request() == isEmpty() then
4:     revert("Request Denied")
5:   else
6:     Get Device_Credentials ( $D\_ID$ ,  $D\_Addr$ )
7:     if Device_Credentials() == alreadyExist() then
8:       revert("Token Not Granted")
9:     else
10:      Device_Registration( $M\_ID$ ,  $D\_ID$ ,  $D\_Addr$ )
11:       $T \leftarrow$  Token_Generation (Hash( $M\_ID$ ,  $D\_ID$ ,  $D\_Addr$ ))
12:       $T_{Sig} = \text{Signature}_{KM-pr}(T)$ 
13:      return  $T_{Sig}$ 
14:      revert("Token Granted")
15:    end if
16:  end if
17: end procedure

```

However, if the token request includes unique credentials of new devices, the miner node successfully registers the devices with its own Identity (M_ID) in the Blockchain. Each new device, with its respective miner node ID, is registered in the Blockchain. Finally, the miner node generates a token that includes unique

credentials such as M_ID , D_ID and D_Addr , signs the token (T_{Sig}) with its private key and sends it to the desired device for authentication. We used the elliptic curve digital signature algorithm (ECDSA) to generate and sign the tokens before sending them to the IoT devices. A private key can be generated by a random seed of 32 bytes in the elliptic curve cryptography algorithm. In our case, we used Ganache's private keys provided by the Ethereum Blockchain network.

3.3.4 Token verification

Algorithm 4 defines a mechanism of token verification and IoT device authentication, in which each IoT device is issued with a signed Blockchain token, T_{Sig} , for secure communication in the IoT network. The token assigned to the IoT device is generated by the miner device with its private key and validated using the public keys of the respective miner nodes that generate the signed token. The public key is derived in the elliptic curve by multiplying the corresponding private key with the generator point of each curve. If the allocated token is successfully validated via Blockchain, the devices are accessed and become part of the IoT network. However, if the token is not successfully authenticated from the Blockchain, the request for access to the network will be denied.

Algorithm 4 Token Verification

Input: Tokens $\leftarrow \{T_{Sig_1}, T_{Sig_2}, T_{Sig_3}, \dots, T_{Sig_n}\}$

Output: Access Granted/Access Not Granted

```

1: procedure DEVICEVERIFICATION
2:   Device_Verification()
3:    $T \leftarrow \text{Verify}_{KM\_pb}(T_{Sig})$ 
4:   if  $T == T_{Sig}$  then
5:     revert("Access Granted")
6:   else
7:     revert("Access Not Granted")
8:   end if
9: end procedure

```

3.4 Execution flow

In a decentralised environment, the authentication of IoT devices is carried out using the various interlinked modules involved in the authentication process. Our Blockchain-based decentralisation scheme includes the following system entities: user, smart contract, miner node, IoT device and authenticated devices which communicate with each other to authenticate the IoT device successfully. Figure 3 shows the execution flow of our proposed model that illustrates the association (successful/failed) between the above system entities. The execution flow demonstrates the flow process for a decentralised authentication scheme according to the system design framework.

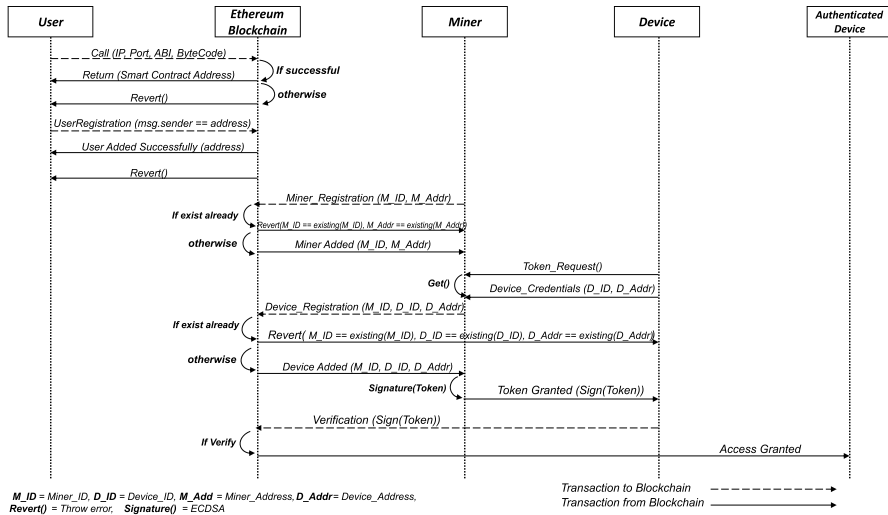


Fig. 3 Execution flow of proposed scheme

The process begins with the user who writes the smart contract in a solidity language first and then compiles and carries out those contracts using the Blockchain addresses accessible at Ethereum Blockchain. The user makes a system call with smart contract parameters such as *IP address*, *port number*, *application binary interface (ABI)* and *byte code*, in order to connect with Ethereum Blockchain. In a successful case, it returns the smart contract address as a further validation address; otherwise, it returns the revert function as an error. When the user retrieves the Ethereum address via the Ethereum Blockchain system call, it enables users to interact and communicate with the Ethereum Blockchain and the other elements of the system in a proper way. The second step is to register the miner nodes to the network by utilising their unique credential, such as M_ID and M_Addr . Such credentials are unique to each of the miner nodes in the system. If the credentials of a new device like M_ID and M_Addr match the credentials of existing miner node, the user is shown an error with the same device credentials. After successfully integrating the miner node into the network, it enables a group of IoT nodes to connect for secure communication.

In the token generation process, the miner node receives a token request from an IoT device that wants to authenticate and communicate with other nodes in a Blockchain-based decentralised network. The token request consists of IoT device credentials such as D_ID and D_Addr which must be unique to others to obtain tokens successfully. Similar to a miner node, the matching function “*alreadyExist()*” is used to check the existence of the IoT device with the same credentials D_ID and D_Addr on the Blockchain. If the credentials for the new IoT devices are already registered in the Blockchain, the error about the existence of the device will be shown to the user. When the IoT device credentials do not match the existing credentials, the miner node attaches an IoT device (M_ID , D_ID and D_Addr) and its corresponding miner’s node ID to build the IoT devices. A miner node can have

several IoT nodes with the same M_ID . The miner node generates a unique token after successful registration of the IoT device and signs it with its key, before being sent to a registered IoT device.

In the last step, IoT devices now connect to the IoT network after the acquisition of the signed token. The Blockchain checks the authenticity of the issuers and signatures, using the respective miner's node public key. Once a Blockchain has successfully validated the token, IoT devices are given access to the authentication network. However, if IoT devices do not provide the correct token, access to the network will not be granted. It is, therefore, easy to find reliable or unsecured devices through the proper issuance of a token.

4 Modelling, analysis and verification of proposed scheme

Formal verification is a systematic approach that uses logical reasoning and evidence to verify the proposed system or algorithm in terms of design and implementation and tests whether or not the proposed system meets the specified requirements [45]. The formal verification process of our proposed scheme consists of three main steps: modelling, analysis and verification. In the modelling step, we modelled the proposed scheme using HLPNs that show the design of the system and its internal relationship with all other components, in detail. In the analysis step, we designed our proposed HLPNs using the PIPE+ tool and then calculated the incidence markings and confidence interval values. In the verification stage, we carried out two activities; one is to define the transition rules using the Z formal specification language; the other is to transform those rules into one of the widely used theories called “the array theory” supported by the SMT-Lib. Regarding the verification results, we performed a bounded model checking technique on our proposed scheme and then verified the algorithms to prove the logic formulae and the satisfiability.

4.1 The high level petri nets

The HLPN provides a way to evaluate system behaviour using modular design and mathematical properties. The benefit of HLPN is that it can be used for the modelling of different systems, including parallel, distributing, synchronous, pseudo-deterministic, concurrent and stochastic systems [46, 47]. Each stated system has its analytical capabilities, as well as its limitations. It is, therefore, important to define fine-grained modularity and generality for protocol analysis and, in particular, for large systems. HLPNs are also referred to as Petri nets and are known as the best method of designing and modelling the system for testing and evaluation purposes. The HLPN can be defined as an array N which consists of seven elements (or tuples) such as $N = \{P, T, F, \varphi, R, L, M_0\}$ in which each individual tuple is defined as follows [48, 49]:

- P is a collection of a finite number of places

- T is a collection of a finite number of transitions, so P and T are two distinct collections represented as $P \cap T = \emptyset$
- F is a directed flow from place to transition or transition to place, so $F \subseteq (P \times T) \cup (T \times P)$
- φ is the mapping function that maps the defined places P to data types such as $\varphi : P \rightarrow \text{Data types}$
- R is a set of finite rules that map T to some degree of the logical formula or reasoning, such as $R : T \rightarrow \text{Formula or Logical reasoning}$
- L is the label mapped on each flow as F , so that $L : F \rightarrow \text{Label}$
- M_0 is the initial state in Petri Net that starts flow and generates tokens $M : P \rightarrow \text{Tokens}$

The three elements (P, T, F) are the basic elements used in the construction of the Petri nets and provide details about their structures. In addition, three elements (φ, R, L) provide information on the Petri net metadata used for implementation and validation. In HLPNs, we firstly describe the places and transitions and then establish the relationship between them. The following illustrations are used to complete the HLPN. For example, places can be depicted as circles, transitions as rectangles and directed arrows used to connect places and transitions and vice versa. However, there can be no relationship between places or transitions. For each HLPN, the “Start” transition used to initialise the HLPN and the “Inputs” places carry the tokens that can enter into the model and transfer to multiple locations and transitions before they reach the end place. The $R(\text{Input}) = \exists x \in X \mid x = \theta$ rule is used to generate tokens and initialise the flow from the input place.

The proposed authentication scheme was divided into four HLPNs based on user registration, miner node registration, token generation and token verification algorithms. Figure 4 demonstrates the HLPN of the process of user registration of the proposed authentication scheme. In Petri nets, the process begins with the “Start” transition, which is considered to be a starting point for any HLPN and is responsible for generating tokens that pass through all other transitions. “Inputs” and “User Registration” places are used to store the initial data (or variables) before and after the transformations are handled, such as the available Ethereum addresses and the registered Blockchain users.

The Petri net of the miner node registration process in our proposed authentication scheme consists of two working transitions and two places, as shown in Fig. 5. Transitions include “Start” and “Check Miner Node Credentials”, while places include “Inputs” and “Miner Node Registration”. “Start” transition initiates the miner node registration process and as well as verifying the miner node credential with the miner node credentials stored in the Blockchain. Additionally, the “Inputs” place include unique credentials such as miner node ID and device addresses related to the miner node. Finally, the registered miner nodes in the proposed scheme are stored at the “Miner Node Registration” place.

Figure 6 displays the HLPN of the token generation process in the proposed scheme. The HLPN includes several transitions, such as “Start”, “Check Token Request”, “Registered IoT Device” and “Token Generation”. On the other side, the places consist of the “Inputs”, “Get Device Credentials”, “Registered IoT Devices”

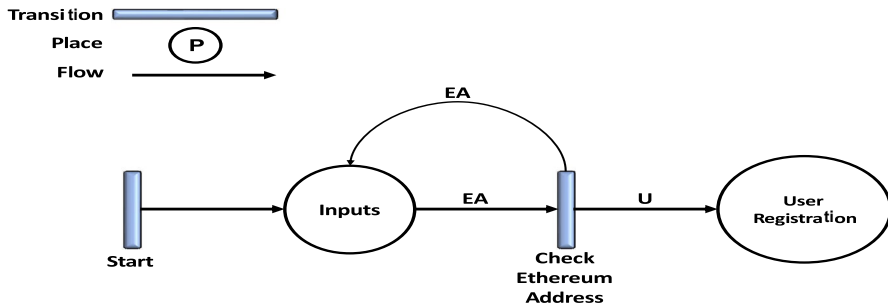


Fig. 4 User registration—Petri Net

and “Token Granted”. The token generation process begins with the “Start” transition, taking unique credentials from users who are linked to the IoT devices and storing them at the inputs site. Following this, the transition called the “Check Token Request” verifies the token request to be sent from the IoT device and stored at the “Get Device Credentials” place. Next, the “Register IoT Device” transition conducts the IoT device registration process with unique credentials such as miner node *ID*, IoT device *ID* and *device address* and stores their credentials in a Blockchain. Finally, the “Token Generation” transition produces a unique token for those IoT devices that successfully pass the entire process and store their tokens in the token granted place.

The HLPN for the token verification function is shown in Fig. 7. The HLPN consists of three transitions, that is, “Start”, “Token Verification”, “Access Checked” and three places that are “Inputs”, “Stored Credentials” and “Access Granted”. The “Start” transition initiates the overall verification process of the token by taking the device credential from the “Stored Credentials” place and by providing access or permission to the system, using the verification mechanism of the token.

The complete HLPN of the proposed authentication scheme is shown in Fig. 8 and the detailed working mechanism is then described using a set of rules from 1–7.

Table 2 explains the places, mappings and description of each place used in the proposed authentication scheme in detail. Since our proposed authentication scheme consists of several steps or procedures, from user registration to token verification, we define different algorithms for each step, as defined in the Sect. 3.3. Each step of the authentication scheme is associated with a different algorithm 1–4 that describes the operation of each component (user, miner node, IoT node) and their interaction with others in the scheme as a whole. In the user registration Petri net, two places are used: inputs and user registration. The inputs place is used to store available Ethereum addresses, while a user registration place is used to store user-related credentials after users have successfully registered with Blockchain. The miner node registration Petri net is divided into two places: inputs and miner node registration, which are used to store miner node attributes. In this case, the inputs place is used to store miner node credentials such as miner ID and miner address. After successfully registering miner nodes with Blockchain, the credentials are stored in the miner node registration place. Furthermore, in the token generation Petri net,

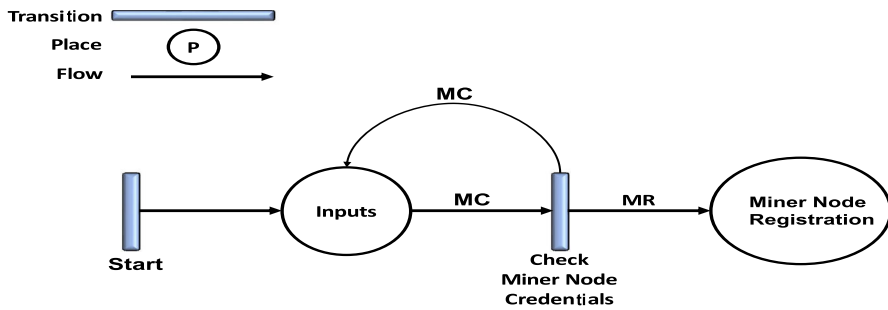


Fig. 5 Miner node registration—Petri Net

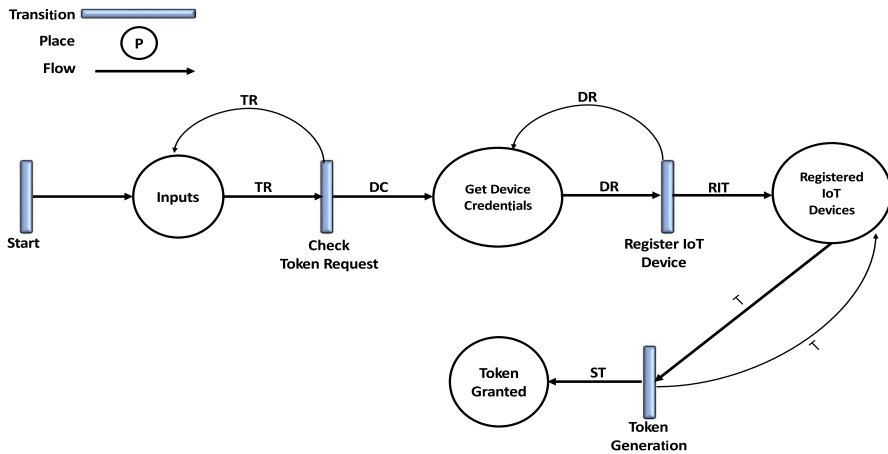


Fig. 6 Token generation—Petri Net

multiple places such as inputs, get device credentials, registered IoT devices and the token granted are combined to generate the tokens. All unregistered device-related credentials are stored in the inputs place. In get device credentials place, the database is used to compare the existing credentials. The registration IoT device place is used to store the registered device credentials. The token granted place is used to store the signed tokens issued to IoT devices as well as the miners' private keys. Finally, the token verification Petri net consists of three places: inputs, stored credentials and access granted. In this case, the inputs place is used to store the miner's public keys as well as signed tokens for verification. All device credentials are saved in the stored credentials place. The access granted place is used to store the final credentials of IoT devices.

The data types used in the proposed scheme and their definition are shown in Table 3. For instance, EA is the byte data type used to represent available addresses in Blockchain. U is a byte data type used in Blockchain to represent the addresses of the registered users. M_{ID} is an integer data type used to represent the miner node's ID. M_{Addr} is a byte data type used to represent the addresses of the miner node. D_{ID}

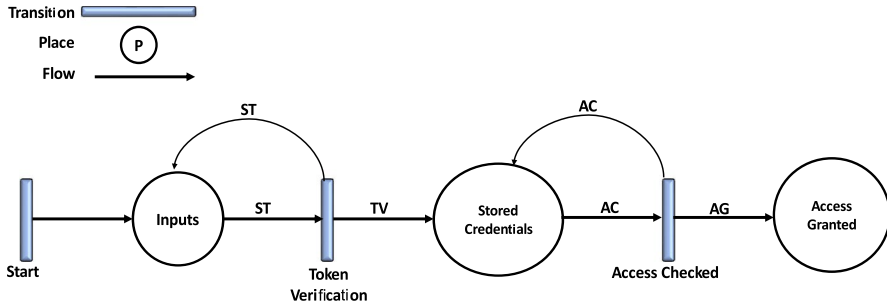


Fig. 7 Token verification—Petri Net

is an integer data type used to represent the IoT device's ID. D_{Addr} is a byte data type used to represent the addresses of the IoT device. T is a byte data type used to represent the tokens allocated to IoT devices. T_{Sig} byte data type used to represent the signed tokens generated for IoT devices. $K_{M_{pr}}$ is byte data type used to represent the private key of a miner node. Finally, $K_{M_{pb}}$ is byte data type used to represent the public key of a miner node.

$$\begin{aligned}
 \mathbf{R}(\text{Check Ethereum Address}) &= \forall ea \in EA, \forall u \in EU | \\
 &[1] := \text{msg.sender}(ea) \wedge \\
 &[2] := \text{ethereum.address}(ea) \wedge \\
 &[3] := \text{Check Ethereum Address}(u[1], u[2]) \wedge \\
 &ea[2] = u[3] \rightarrow \} \} \text{Verified}' \wedge U' = U \cup \{u\} \vee \\
 &ea[2] \neq u[3] \rightarrow \} \} \text{Not} - \text{Verified}' \wedge U' = U \cup \{u\}
 \end{aligned} \tag{1}$$

The authentication process begins with the registration of the user in the system when the users register with the Ethereum available addresses provided by the Blockchain. Generally, the “msg.sender” feature of the smart contract returns the address of the smart contract user in the Ethereum Blockchain. The test Ethereum function compares the address of the registered user with the “msg.sender” as shown in Rule 1.

$$\begin{aligned}
 \mathbf{R}(\text{Check Miner Node Credentials}) &= \forall mc \in MC, \forall mr \in MR | \\
 &mr[1] := \text{miner id}(mc) \wedge \\
 &mr[2] := \text{miner address}(mc) \wedge \\
 &mr[3] := \text{Check Miner Node Credentials}(mr[1], mr[2]) \wedge \\
 &mc[2] = mr[3] \rightarrow \} \} \text{Verified}' \wedge MR' = MR \cup \{mr\} \vee \\
 &mc[2] \neq mr[3] \rightarrow \} \} \text{Not} - \text{Verified}' \wedge MR' = MR \cup \{mr\}
 \end{aligned} \tag{2}$$

Rule 2 explains the procedure for registering the miner node in the Blockchain network. The user registers the miner node to the system with unique data such as the miner node ID and the device address. The “check miner node credential” function is used to verify the existence of already stored credentials on the Blockchain.

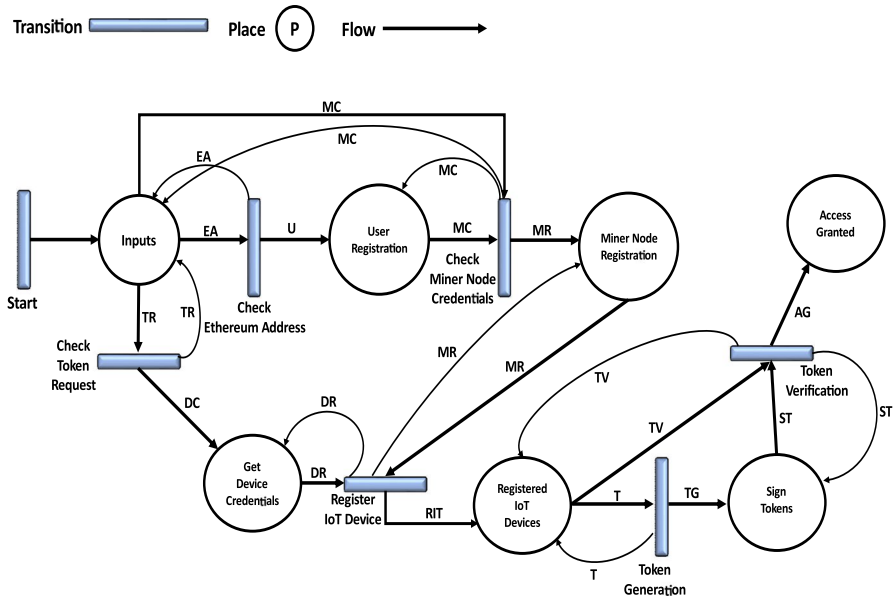


Fig. 8 A complete HLPN of the TokenAuth scheme

Upon successful execution, the function returns the existence or non-existence of the already registered miner node to the user.

$$\begin{aligned}
 R(\text{Check Token Request}) &= \forall tr \in TR, \forall dc \in DC | \\
 &\quad dc[1] := \text{miner id}(tr) \wedge \\
 &\quad dc[2] := \text{device id}(tr) \wedge \\
 &\quad dc[3] := \text{device address}(tr) \wedge \\
 &\quad dc[4] := \text{Check Token Request}(dc[1], dc[2], dc[3]) \wedge \\
 &\quad DC' = DC \cup \{dc[1], dc[2], dc[3], dc[4]\}
 \end{aligned} \quad (3)$$

In Rule 3, the function called “check token request” verifies the request token from the user who wants to connect the IoT device to the network. The request for a token is checked against unique credentials such as miner node ID, device ID and device address for the new IoT device added to the Blockchain.

$$\begin{aligned}
 R(\text{Register IoT Device}) &= \forall dr \in DR, \forall mr \in MR, \forall rit \in RIT | \\
 &\quad rit[1] := dr[1] \wedge \\
 &\quad rit[2] := mr[1] \wedge \\
 &\quad rit[3] := \text{Register IoT Device}(rit[1], rit[2]) \wedge \\
 &\quad RIT' = RIT \cup \{rit[1], rit[2], rit[3]\}
 \end{aligned} \quad (4)$$

Upon checking the request for the token and verifying the credential from the Blockchain, the new IoT device is added to the Blockchain, as shown in Rule 4.

Table 2 Places, mappings and description of TokenAuth scheme

Places	Mappings	Description
<i>User Registration</i>		
φ (Inputs)	$P(EA)$	Holds the available ethereum addresses
φ (User Registration)	$P(U)$	Store registered users integrated with the TokenAuth scheme
<i>Miner Node Registration</i>		
φ (Inputs)	$P(M_ID \times M_Addr)$	Holds the credentials of miner node such as miner ID and miner address
φ (Miner Node Registration)	$P(U \times M_ID \times M_Addr)$	Stores the miner node credentials with the registered user
<i>Token Generation</i>		
φ (Inputs)	$P(M_ID \times D_ID \times D_Addr)$	Holds the device credentials such as miner ID, device ID, and device address
φ (Get Device Credentials)	$P(D_ID \times D_Addr)$	Check device credentials from database
φ (Registered IoT Device)	$P(M_ID \times D_ID \times D_Addr)$	Stored device credentials with miner ID
φ (Token Granted)	$P(T \times T_{sig} \times K_{M_{pr}})$	Stores signed tokens and miner private keys
<i>Token Verification</i>		
φ (Inputs)	$P(T_{sig} \times K_{M_{pr}})$	Holds miners public keys and signed tokens
φ (Stored Credentials)	$P(M_ID \times D_ID \times D_Addr)$	Stored device credentials
φ (Access Granted)	$P(T_{sig} \times T)$	Access Granted to device

Table 3 Data types used in proposed scheme

Data Types	Description
EA	A byte type for the representation of available addresses in Blockchain
U	A byte type for the representation of addresses of registered users
M_ID	An integer type for the representation of ID of miner node ID
M_Addr	A byte type for the representation of the address of the miner node
D_ID	An integer type for the representation of ID of IoT device
D_Addr	A byte type for the for the representation of address of IoT device
T	A byte type for the representation of tokens allocated to IoT devices
T _{sig}	A byte type for the representation of signed tokens to IoT devices
K _{M_{pr}}	A byte type used to represent the private key of the miner nodes
K _{M_{pb}}	A byte type used to represent the public key of the miner nodes

$$\begin{aligned}
\mathbf{R}(\text{Token Generation}) &= \forall t \in T, \forall st \in ST | \\
&\quad st[1] := \text{private key } (t) \wedge \\
&\quad st[2] := \text{hash } (t) \wedge \\
&\quad st[3] := \text{Token Generation}(st[1], st[2]) \wedge \\
&\quad ST' = ST \cup \{st[1], st[2], st[3]\}
\end{aligned} \tag{5}$$

Rule 5 defines the process of the generation of tokens for IoT device authentication in the system. The system generates a token for the IoT device after checking its credentials from the Blockchain. Upon successful verification, the miner node signs the token with its private key and sends it to the IoT device for the authentication process.

$$\begin{aligned}
\mathbf{R}(\text{Token Verification}) &= \forall tg \in TG, \forall st \in ST, \forall tv \in TV | \\
&\quad tv[1] := st[1] \wedge \\
&\quad tv[2] := \text{public key } (tg) \wedge \\
&\quad tv[3] := \text{Token Verification}(tv[1], tv[2]) \wedge \\
&\quad TV' = TV \cup \{tv[1], tv[2], tv[3]\}
\end{aligned} \tag{6}$$

Rule 6 defines the process of verification of tokens already issued to the IoT device in Rule 5. During the verification process, the token given to the IoT system is verified with the public key of the miner node.

$$\begin{aligned}
\mathbf{R}(\text{Access Checked}) &= \forall ac \in AC, \forall ag \in AG | \\
&\quad ag[1] := \text{Token } (ac) \wedge \\
&\quad ag[2] := \text{signed Token } (ac) \wedge \\
&\quad ag[3] := \text{Access Checked}(ag[1], ag[2]) \wedge \\
&\quad ac[2] = ag[3] \rightarrow \} \} \text{Access Granted''} \wedge AG' = AG \cup \{ag\} \vee \\
&\quad ac[2] \neq ag[3] \rightarrow \} \} \text{Access Not Granted''} \wedge AG' = AG \cup \{ag\}
\end{aligned} \tag{7}$$

Finally, in Rule 7, after successful verification of the token, the IoT device credentials are compared with those credentials which are already stored on the Blockchain. If the access check function is successfully executed, the device is then activated to access the system resource for additional system options.

4.1.1 Analysis

We model our proposed authentication scheme using one of the popular modelling tools named PIPE+, which is widely used for the implementation and analysis of the Petri nets built under bounded model checking (BMC). The formal modelling of HLPNs ensured that our proposed scheme was ultimately executable for the specified rules and parameters. To analyse our model, we obtained simulation results based on two evaluation parameters, such as incidence marking and confidence intervals (minimum and maximum threshold values).

4.1.2 Incidence marking

We calculate the different types of incidence marking matrices, such as forward, backward, combined and inhibition, which are used to describe the relationship between places and transitions. Forward and backward incidence matrices are used to define the relationship between places and transition in HLPNs. The arc from position P to transition T only leads to value $(P, T) > 0$ if the desired input is transferred from place to transition. The forward incidence matrix represents the total number of tokens generated from the places and fires towards the transition. In contrast, the backward incidence matrix represents the total number of tokens available at the place P , used to allow the transaction T . In forward and backward incidence matrices, the 1s show the interactions between places and transitions and the 0s show no interaction between places and transitions.

The combined incidence matrix I combines the incidence results, such as 0s and 1s of both forward and backward incidence matrices, in order to provide an overview of the token generated at different locations and to find the difference between them. Finally, we calculated the inhibition matrix H of the Petri Nets given, which describes the weak relation between the places in the given graph, if any. In our case, the 0s in the inhibition matrix indicate the connectivity of all places to others, confirming the connectivity and reachability of the places.

These matrices are used to improve the visibility of Petri nets and enhance the confidence level of any model in relation to verifiability and correctness. Moreover, these matrices tend to offer the best accuracy and reliability of the results of any protocol or method. We use the PIPE+ editor to calculate and provide the matrices above, for each Petri net included in the proposed authentication scheme.

Table 4 displays the results of forward, backward, combined and inhibition matrices for the scenario of user registration. For the forward incidence matrix, the place is the transition's output location, while for the backwards incidence matrix, the place is the transition's input location. If no relation exists, the matrix entry is 0. As we can see, the flow of the "Start" transition is towards the "Inputs" place, which is represented by 1 and it has no direct relation to the "Check Ethereum Address"

transition, which is represented by 0. Similarly, the “User Registration” place receives the transition flow from the “Check Ethereum Address” transition and is depicted in 1. For backward incidence matrix, the “Inputs” place is only connected to the “Check Ethereum Address” transition. Other places and transitions, on the other hand, have no relationship with one another and are thus represented by 0s. In the case of a combined incidence matrix, -1 represents the backwards relation direction from the transition to the place. In this Petri net, for example, the “Check Ethereum Address” is flowing towards the back “Inputs” place and is thus defined by -1 . The 1 depicts a direct link from transition to place, such as flow from the “Start” transition to the “Inputs” place. However, 0 indicates that there is no relation between the transition and the place. As previously mentioned, the inhibition matrix defines the weak relationship, if any, between the places in the given table. In our case, the 0s in the inhibition matrix show the connectivity of all places to others, confirming the places’ connectivity and reachability.

The results of forward, backward, combined and inhibition incidence matrices for the miner node registration Petri net are shown in Table 5. As can be seen, the flow of the “Start” transition is towards the “Inputs” place, which is represented by 1 and it has no direct relation to the “Check Miner Node Credentials” transition, which is represented by 0. Similarly, the “Miner Node Registration” place receives the transition flow from the “Check Miner Node Credentials” transition and is shown in 1. For backwards incidence matrix, the “Inputs” place is only connected to the “Check Miner Node Credentials” transition. Other places and transitions, on the other hand, have no relationship with one another and are thus represented by 0s. For the combined incidence matrix, the “Check Miner Node Credentials” value is -1 because it is flowing back towards the “Inputs” place. The 1 depicts a direct link from transition to place, such as flow from the “Start” transition to the “Inputs” place. However, 0 indicates that there is no relation between the transition and the place. In our case, the 0s in the inhibition matrix shows the connectivity of all places to others, confirming the places’ connectivity and reachability.

Table 6 shows the results of forward, backward, combined and inhibition matrices for the token generation Petri net. As we can see, the flow of the “Start” transition is towards the “Inputs” place, which is represented by 1 and it has no direct relation to the other transitions such as “Check Token Request”, “Register IoT Device” and “Token Generation”, which are represented by 0. Similarly, the flow of the “Check Token Request” is towards the “Get Device Credentials” transition, which is represented in 1 and it has no direct connection to the other transitions such as “Start”, “Register IoT Device” and “Token Generation”, which are represented in 0. Furthermore, transitions such as “Register IoT Device” and “Token Generation” flow from “Registered IoT Device” and “Token Granted” places. In the backward incidence matrix, the “Inputs” place is associated with the “Check Token Request” transition and is represented by 1; it has no direct relation to the other places. Similarly, the “Get Device Credential” place is only associated with the “Register IoT Device”. The transitions from “Registered IoT Device” to “Token Generation” is related. Finally, the “Token Granted” place is not connected to any other place in the Petri net, so it displays all 0s. For the combined incidence matrix, the following flows from transitions to places occur, such as from “Check Token Request” to “Inputs”,

Table 4 High level Petri Net incidences—User Registration

	Start	Check Ethereum Address		Start	Check Ethereum Address
<i>Forwards incidence matrix I^+</i>			<i>Backwards incidence matrix I^-</i>		
$\phi(\text{Inputs})$	1	0	$\phi(\text{Inputs})$	0	1
$\phi(\text{User Registration})$	0	1	$\phi(\text{User Registration})$	0	0
<i>Combined incidence matrix I</i>			<i>Inhibition matrix H</i>		
$\phi(\text{Inputs})$	1	-1	$\phi(\text{Inputs})$	0	0
$\phi(\text{User Registration})$	0	1	$\phi(\text{User Registration})$	0	0

from “Register IoT Device” to “Get Device Credentials” and from “Token Generation” to “Registered IoT Devices”, all of which are expressed in -1 . The 1 depicts a direct link from transition to place, such as flow from the “Start” transition to the “Input” place. However, 0 indicates that there is no relation between the transition and the place. In our case, the 0s in the inhibition matrix shows the connectivity of all places to others, confirming the places’ connectivity and reachability.

Forward, backward, combined and inhibition incidence matrices for the token verification scenario are shown in Table 7. As can be shown, the flow of the “Start” transition is towards the “Inputs” place, which is represented by 1 and it has no direct relationship to the “Token Verification” and “Access Checked” transitions, which are represented by 0. Similarly, the “Token Verification” transition received the transition flow from the “Inputs” place and stores the credentials in the “Stored Credentials” place and represented 1. Finally, the “Access Checked” transition only receives the flow from “Stored Credentials” and directs it to the “Access Granted” place, which is denoted by 1. The “Inputs” place in the backwards incidence matrix is linked to the “Token Verification” transition and thus represented in 1. The “Access Checked” transition is correlated with the “Stored Credentials” place. The “Access Granted” place, on the other hand, has no relationship to the transitions in the Petri net and is thus represented by 0s. The following flows from transition to place occur in the combined incidence matrix, such as “Token Verification” to “Inputs” and “Access Verified” to “Stored Credentials” and are thus all represented in -1 . The 1 represents a direct connection from one transition to another, such as flow from the “Start” transition to the “Inputs” place. However, a value of 0 means that no relationship exists between the transitions and the places. In our case, the 0s in the inhibition matrix demonstrates the connectivity of all places to others, confirming the connectivity and reachability of the places.

4.1.3 Confidence interval

For a confidence interval, the minimum and maximum threshold values for each Petri net are calculated in the proposed authentication scheme. Minimum threshold values are calculated by passing 100 firings and five replications to the Petri nets and the maximum threshold value is calculated by passing 1000 firings and 50

Table 5 High level Petri Net incidences—Miner Node Registration

	Start	Check Miner Node Creden- tials		Start	Check Miner Node Cre- dentials
<i>Forwards incidence matrix I^+</i>			<i>Backwards incidence matrix I^-</i>		
$\phi(\text{Inputs})$	1	0	$\phi(\text{Inputs})$	0	1
$\phi(\text{Miner Node Registra-tion})$	0	1	$\phi(\text{Miner Node Registra-tion})$	0	0
<i>Combined incidence matrix I</i>			<i>Inhibition matrix H</i>		
$\phi(\text{Inputs})$	1	-1	$\phi(\text{Inputs})$	0	0
$\phi(\text{Miner Node Registra-tion})$	0	1	$\phi(\text{Miner Node Registra-tion})$	0	0

replications. The simulation results show the minimum and maximum threshold values calculated for the 95% confidence interval and presented in terms of the average number of tokens, which are produced at each place, as well as the acceptable level of error during each processing state.

The confidence interval is derived from the probability value of each processing state and reflects the confidence range from 1 to α . The confidence interval is also expressed as a percentage and informs the reader of the occurrence of accurate findings within the given range. In our simulation results, we set the parameter $\alpha = 0.05$ to determine the minimum and maximum threshold, using $(1-\alpha)$ which gives a 95% confidence value in terms of the probability of accurate results.

In our HLPNs, the confidence interval values is determined by the average number of tokens generated at each transition T and the positive value represents the highest confirmation of the accuracy of the proposed authentication scheme. Figures 9, 10, 11 and 12 indicate the minimum and maximum threshold values for each Petri net under the proposed scheme.

Figure 9 depicts the confidence interval of a user registration scenario calculated using the minimum and maximum threshold values. As previously mentioned, we determined the minimum threshold values of the user registration Petri net by passing 100 firings and five replications. We pass 1000 firings and 50 replications to the Petri net for user registration for maximum threshold values. As the “Inputs” place includes all of these credentials for registering a user on the Blockchain, the minimum and maximum thresholds exhibit a confidence interval between two and twenty on the confidence interval scale. The “user registration” place, on the other hand, only contains the credentials of registered users, with their minimum and maximum thresholds displaying a confidence interval between zero and ten on the confidence interval scale.

Figure 10 depicts the confidence interval for a miner node registration scenario on the Blockchain. In the miner node registration Petri net, two places are used: inputs and miner node registration. Since the “Inputs” place stored the credentials of the miner node that will be registered on Blockchain, the minimum and maximum threshold values measured on the confidence interval scale are three and 19,

Table 6 High level Petri Net incidences—Token Generation

	Start	Check Token Request	Register IoT Device	Token Generation
<i>Forwards Incidence Matrix I^+</i>				
φ (Inputs)	1	0	0	0
φ (Get Device Credentials)	0	1	0	0
φ (Registered IoT Devices)	0	0	1	0
φ (Token Granted)	0	0	0	1
<i>Backwards Incidence Matrix I^-</i>				
φ (Inputs)	0	1	0	0
φ (Get Device Credentials)	0	0	1	0
φ (Registered IoT Devices)	0	0	0	1
φ (Token Granted)	0	0	0	0
<i>Combined Incidence Matrix I</i>				
φ (Inputs)	1	-1	0	0
φ (Get Device Credentials)	0	1	-1	0
φ (Registered IoT Devices)	0	0	1	-1
φ (Token Granted)	0	0	0	1
<i>Inhibition Matrix H</i>				
φ (Inputs)	0	0	0	0
φ (Get Device Credentials)	0	0	0	0
φ (Registered IoT Devices)	0	0	0	0
φ (Token Granted)	0	0	0	0

Table 7 High level Petri Net incidences—Token Verification

	Start	Token Verification	Access Checked
<i>Forwards Incidence Matrix I^+</i>			
φ (Inputs)	1	0	0
φ (Stored Credentials)	0	1	0
φ (Access Granted)	0	0	1
<i>Backwards Incidence Matrix I^-</i>			
φ (Inputs)	0	1	0
φ (Stored Credentials)	0	0	1
φ (Access Granted)	0	0	0
<i>Combined Incidence Matrix I</i>			
φ (Inputs)	1	-1	0
φ (Stored Credentials)	0	1	-1
φ (Access Granted)	0	0	1
<i>Inhibition Matrix H</i>			
φ (Inputs)	0	0	0
φ (Stored Credentials)	0	0	0
φ (Access Granted)	0	0	0

respectively. The “miner registration” place, on the other hand, stores the data of the only registered miner nodes on Blockchain and displays the minimum and maximum values between one and ten on the scale.

Figure 11 depicts the confidence interval for a token generation scenario for IoT devices. In the token generation Petri net, four places are utilized: inputs, get device credentials, registered IoT devices and token granted. Since the “Inputs” place stored the credentials of both miner nodes and IoT devices, the minimum and maximum threshold values calculated on the confidence interval scale are between three and twenty, respectively. The “get device credentials” place retrieves the data and stores the IoT device credentials and displaying the minimum and maximum values between one and ten on a scale. Similarly, the scale values for “registered IoT devices” place range from 1 to 7. Finally, the “token granted” place stores the signed token of the IoT devices and displays values ranging from 2 to 8 on a scale.

Figure 12 depicts the confidence interval for a token verification scenario for IoT devices. Three places are used in the token verification Petri net: inputs, stored credentials and access granted. Since the “Inputs” place stored the credentials of both miner nodes and IoT devices to verify the registered IoT devices to the Blockchain, the minimum and maximum threshold values calculated on the confidence interval scale are between eight and 27, respectively. The “stored credentials” place store the IoT device-related credentials such as signed tokens and therefore displaying the minimum and maximum values between seven and 25 on a scale. Similarly, the scale values for “access granted” place range from 3 to 15.

The simulation results obtained from the maximum and minimum threshold values for HLPNs show the precision of the proposed scheme in terms of the accessibility of different places and states using the specified rules.

4.2 Formal verification

To verify and prove the correctness of any protocol or underlying system, verification is a commonly used method that demonstrates and checks the validity of the system against specified requirements and other parameters. These requirements and parameters can be specified for each system state and represent their internal procedure or mechanism. For verification, system modelling and interpretation, as well as automatic logic and decision-making problems, a variety of solvers are available that offer an extensive range of functions for system verification. One such type of solver is the Boolean Satisfiability (SAT) [50] solver which provides users with feedback on system evaluation and conformity with given proposition logic formulae using Boolean predicates or statements. In simpler terms, Boolean Satisfiability is a process by which a Boolean formula is assessed as being satisfactory or not. Generally, the SAT used to solve nondeterministic polynomial-time complete (NP completeness) problems means that there are no polynomial-time algorithms available to solve SAT problems. However, the drawback of the Boolean Satisfiability solver is that it only solves decision problems with Boolean logic, in which complexity can be scaled up with system-level difficulties and therefore cannot extend to a widely distributed network [51].

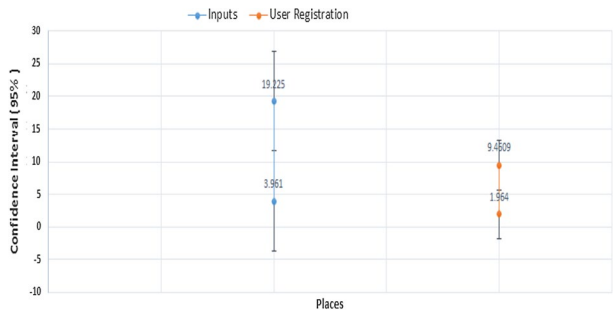


Fig. 9 Minimum and maximum threshold values (User Registration)

Fig. 10 Minimum and maximum threshold values (Miner Node Registration)

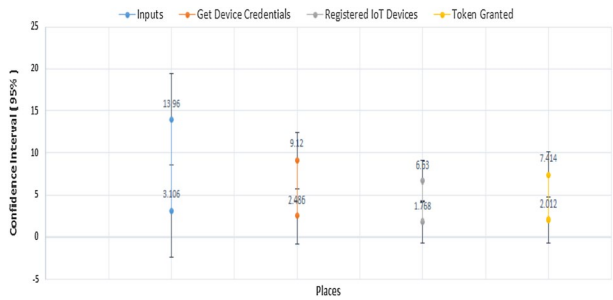
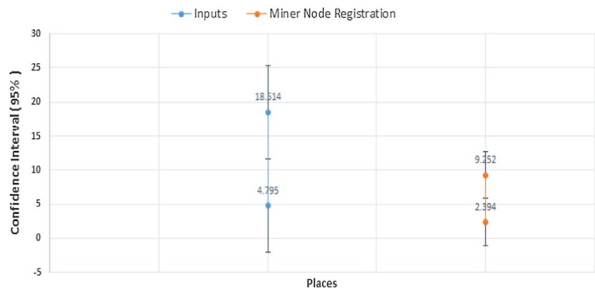


Fig. 11 Minimum and maximum threshold values (Token Generation)

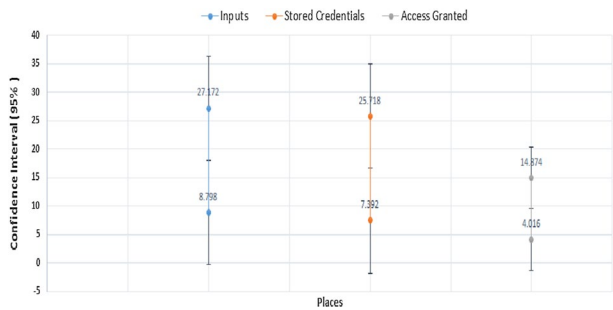


Fig. 12 Minimum and maximum threshold values (Token Verification)

To solve large decision-making problems, Satisfiability Modulo Theories [45] is a widely used decision process that solves multiple problems based on first-order logic formulae and provides system satisfaction, using a wide variety of dynamically typed theorems. The benefit of SMT is that it can support a wide variety of theories in conjunction with multiple decision problem ranges, such as linear and non-linear problems with integers, reals, rationales, arrays and even supported equality, as well as bit vectors and uninterrupted functions [52]. To circumvent these theories and functions, SMT-Lib is used to provide the verification facility for many other solvers. In SMT-Lib, the first behavioural requirements of the proposed system specified the use of abstract models and then bounded model tests were performed using bounded symbolic execution. SMT-Lib is a library developed by Microsoft Research, which can support or incorporate a variety of solvers, such as [53], Boolector [54], Beaver [55] and Z3 [56].

We used the Z3 SMT solver to verify our proposed authentication scheme. Z3 SMT Solver is considered to be an efficient solver and widely used for the evaluation and analysis of algorithms, applications or complete systems under one or more theories [56]. From the above theories, we used array theory to validate and fulfill our logic rules (or formulae) and specifications, as described in our proposed scheme.

In order to verify the proposed authentication scheme, we have specified the following properties that represent our designed algorithms for the proposed scheme.

- *User Registration* The property specified for the user registration algorithm is shown in Rule 8 in which users are registered with the available Ethereum addresses provided by the Blockchain. The comparison function is used to compare the available Ethereum addresses with the msg.sender function for the registration of the user to the Blockchain.

$$\begin{aligned} &(\text{assert}(\text{not}(\text{or}(\text{= }(\text{select msg.sender } 1) \\ &(\text{select ethereum.address } 2)))))(\text{check-sat}) \end{aligned} \quad (8)$$

- *Miner Node Registration* In the miner node registration property, the attributes associated with the miner node are selected and asserted in the system. If the asserted attributes are already stored in the system, the device will not be able to register with the Blockchain. The property for the registration of the miner node is shown in Rule 9.

$$\begin{aligned} &(\text{assert}(\text{not}(\text{or}(\text{= }(\text{select miner id } 1) (\text{select} \\ &\text{miner address } 2) (\text{= }(\text{select miner id verified}) \\ &(\text{select miner address verified})))))))(\text{check-sat}) \end{aligned} \quad (9)$$

- *Token Generation* The property for the generation of a token is shown in Rule 10. In this property, a token is generated for the registration and authentication of IoT devices on the Blockchain system. This token is generated and signed by the respective miner nodes. If the credential for IoT devices is already registered with the relevant miner nodes, the token will not be issued to IoT devices.

$$\begin{aligned} & (\text{assert}(\text{not}(\text{or}(= (\text{select miner id } 1) (\text{select} \\ & \text{device id } 2) (\text{select device address } 3) (= (\text{select} \\ & \text{hash})(\text{select private key })))))) (\text{check-sat}) \end{aligned} \quad (10)$$

- *Token Verification* In the token verification property, the task is to validate the token issued to the IoT device before adding and providing access to the system. In this step, the token assigned to the IoT device is verified correctly by the system. The IoT device is allowed to communicate with the system and other devices based on verified tokens. The property for verification of the token is shown in Rule 11.

$$\begin{aligned} & (\text{assert}(\text{not}(\text{or}(= (\text{select tv } 1) (\text{select public key } 2)) \\ & (= (\text{select token})(\text{select signed token})))) (\text{check-sat}) \end{aligned} \quad (11)$$

–

4.2.1 Results

In the verification phase, both the proposed model and the transition rules or properties are defined and converted to SMT. We used the logic of “QF AUFLIA” available in SMT-Lib to transform the rules. This logic offers quantified free logic formulae and free sort functions that can be easily incorporated into various theories, such as integer arrays. Further, Z3 solver is then used to test whether or not the proposed model satisfies the specified properties. The results of the system modelling and the specified properties imply the correctness of the proposed scheme as regards validity and reliability in the underlying scenario. The Z3 solver takes the stated properties as an input parameter, performs the computation on them and then produces the results in the Boolean satisfiers form, such as “sat” and “unsat”. Unsatisfactory results are shown as “sat” and lead to imprecision or violation of the specified properties, while “unsat” results are shown as leading to the verifiability and correctness of the proposed scheme in terms of system modelling and properties.

The verification results of the proposed authentication scheme are obtained in terms of the execution time that the solver takes and spends on the verification of the specified properties of the proposed algorithms. The execution time (in seconds) of the specified properties is shown in Table 8. The verification results of the specified properties are carried out in a finite time and provide confirmation and satisfaction of the properties in terms of correctness. The listed verification properties such as user registration and miner node registration take 0.01 second to execute on the Z3 solver, as shown in the table. The token generation property takes slightly longer than the other properties because it takes the credentials of both the miner node and the IoT device and then generates a token for the IoT device. Finally, the token verification property verifies the results in 0.02 seconds.

5 Evaluation framework

Our evaluation framework consists of various implementation paradigms such as *Ethereum Blockchain*, *Elliptic Curve Cryptography*, *Python*, *Web3.py*, *Ganache*, *JSON-RPC*, *Solidity* and *Truffle Suite*, as shown in Fig. 13. These implementation paradigms help us to develop the underlying functions of our proposed solution and to evaluate them efficiently. The details about these paradigms are as follows: for example, we used the Ethereum Blockchain to implement our proposed decentralised authentication prototype because it is a commonly used platform for building decentralised apps (dApps) and it provides a secure way to perform transactions using the elliptic curves cryptography protocol.

In addition, the Ethereum Blockchain facilitates the process of deploying and running smart contracts that are an important part of our proposed authentication mechanism. We created our prototype interfaces (user, miner node, simple IoT device) using the Python language, which is considered a dynamic and scalable language across multiple platforms. We used the Web3.py library, which is a Python-based library and enabled us to interact with Ethereum clients to request functions written in smart contracts. The Ganache is a Blockchain emulator (a personal Ethereum client or node) to test a decentralised application without the actual set-up of the Ethereum network. JSON-RPC is a client-side library or communication protocol intended for JavaScript application communication. Our smart contract functions, enabling the miner nodes to authenticate and validate IoT devices securely, are based on solidity language. Finally, we use the Truffle Suite to write, deploy and execute smart contracts in the Ethereum Blockchain.

6 Performance evaluation

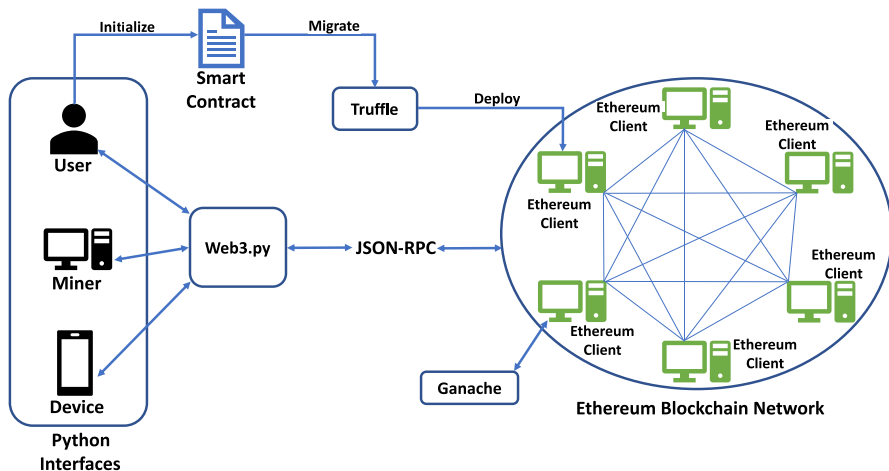
In performance evaluation, we compare firstly the computational and communication overhead of the proposed system with the current authentication schemes based on Blockchain [31, 36, 37] and then measure the approximate financial cost for using Ethereum Classic (ETC) cryptocurrency to execute our scheme on a public Blockchain. As an example for the evaluation, we consider an IoT-based smart home case. Our experiments are performed on a system with the requirements shown in the Table 9. We conducted our experiments on 64-bit Ubuntu 18.4 operating systems. The central processing unit (CPU) architecture used in the 64-bit operating system was x86_64 with a clock cycle of 2.11 GHz. Our experimental specification included 8 GB of RAM.

6.1 A case study of IoT based smart home

A smart home system is made up of a range of linked smart home appliances, with the IoT serving as a key enabler in the smart home environment, allowing for home automation and improving quality of life. In an IoT-based smart home,

Table 8 Execution time of the specified properties

Specified properties	Execution time (sec)
User Registration	0.01
Miner Node Registration	0.01
Token Generation	0.03
Token Verification	0.02

**Fig. 13** Evaluation framework

smart devices, also referred to as IoT devices, are installed across the house to provide the homeowner with a secure environment. For instance, security alarms and motion detectors equipped with intelligent functions can operate intelligently and alert the homeowner in the event of a security breach occurred. However, the security of smart home devices is a significant concern, as hackers often target IoT devices, the protection of smart home devices is a major concern. These devices must be authenticated to guarantee the confidentiality and integrity of data transmitted via the IoT network. The absence of advanced authentication measures on IoT devices is often overlooked, allowing intruders to deploy unregistered devices and compromise existing devices in order to gain complete control of the home. As a result, adequate authentication of these smart home-enabled IoT devices becomes increasingly critical, as unauthorised access can result in resource mismanagement, home invasion attacks, or the disclosure of private information.

As IoT-based smart home applications are becoming increasingly popular for complete home automation and security, we demonstrate the applicability of our proposed Blockchain-based authentication scheme in an IoT-based smart home scenario. An IoT-based smart home scenario in which a user deploys miner nodes

Table 9 System specifications

Operating systems	Operating systems mode	CPU architecture	CPU clock cycle	RAM
Ubuntu 18.04	64-bits	x86_64	2.11 GHz	8 GB

for validating and authenticating IoT devices, in order to test the efficiency of our proposed authentication scheme. In our case, miner nodes can be powerful devices (mobile phones and laptops) for various computing tasks, such as creating public/private keys and tokens. However, IoT devices are resource-constrained devices with low power, battery and memory and therefore must authenticate using an Ethereum Blockchain. IoT devices can include thermostat, video cameras, air monitoring devices, security alarms and smoke detectors.

We consider three different cases based on miner nodes and IoT devices in our performance evaluation scenario. In all three cases, the miner nodes are controlling and managing 3, 5 and 10 different IoT devices, respectively.

6.2 Computational overhead

Computational overhead or computational time is the time required for carrying out different system operations. The computational overhead is split into two main categories, namely on-chain and off-chain, for Blockchain IoT applications. The operations performed on Blockchain are included in the category of on-chain operations, such as initialisation and association of IoT devices and miner nodes on the Blockchain. On the other hand, off-chain operations are local operations, which are not related to Blockchain operations, for example, the generation of public and private keys and tokens. In this paper, we only consider on-chain operations because the costs of off-chain operations vary from system to system, depending on the system specification. We conducted each operation 100 times and then collected the average time (in seconds) of each operation for accurate results of operations.

We first initialise miner nodes on the network, following the IoT-based smart home scenario. The user creates a transaction to add the first miner node with a miner's ID and device address and sends it to the Ethereum Blockchain. The user also initialises the other miner nodes by sending the transactions to Blockchain. If the miner node is already on the Blockchain with the same information, the transaction is discarded. We then calculate the execution time for each transaction on the Blockchain, after the deployment of the miner's nodes. We compare the computation overhead of initialising the miner nodes (or devices) to the proposed schemes with the current Blockchain-based authentication schemes [31, 36, 37] and concluded that our method generates less overhead in the system. The analysis of the computation overhead of our proposed scheme with the current Blockchain schemes is revealed in Fig. 14(a).

We begin our process in the second step with the integration of IoT devices. As indicated in our case study scenario, each miner node can handle many IoT devices. Once a token request is received from the IoT device, the miner node creates a transaction

with the miner ID, device ID and device address and transmits it to the Blockchain. A comparison between our scheme and existing schemes to deploy IoT devices on the network is shown in Fig. 14(b). Finally, the overall statistics for the deployment of the miner nodes and IoT device in the system are illustrated in Fig. 14(c). In contrast to the current Blockchain-based systems [31, 36, 37], our proposed system achieves significant computation performance and takes less time in processing and mining transactions for the initialisation of miner nodes and IoT devices on the Blockchain.

6.3 Communication overhead

In communication overhead, the two communication scenarios are considered: (i) between the user and miner nodes and (ii) between miner nodes and IoT devices. In each case, the overhead measurement depends on the number of transactions and the parameters used for each transaction to customise Blockchain tools. In each case, the overhead communication depends on the number of transactions and the parameters used to initialise the devices on Blockchain. We measure the duration of each transmission to and from the Blockchain. We measure each transaction time at least 100 times in the computation overhead and then obtain the average value for precise results.

In the former scenario, we calculate the communication cost of adding miner nodes to the Ethereum Blockchain, in which the user transmits a transaction containing unique credentials to the Blockchain and then returns the association's transactions for purposes of comparison with the Blockchain. The computation overhead of adding miner nodes to the Blockchain is shown in Fig. 15(a). Simultaneously, for the latter scenario, we send and receive initialisation and association transactions (requests) for various IoT devices under different miner nodes and then calculate the communication cost, as shown in Fig. 15(b). An analysis of results shows that our system achieves communication efficiency and interaction with different IoT devices in the Blockchain system. Fig. 15(c) describes the overall communication overhead through both the initialisation and association transactions for all miner nodes and IoT devices. In each case, we compare the proposed scheme with existing authentication systems based on Blockchain [31, 36, 37] and conclude that our scheme is also effective in communication overhead, taking less time for communication between IoT devices and Blockchain set-up.

6.4 Financial cost

We estimate the approximate financial cost of using the Ethereum network for implementing an IoT based smart home. Given that our scheme is based on a public Blockchain, we must pay the price (in gas) to validate and store all transactions in Blockchain. This cost is determined using an Eq. 12 in [31] and the cost for a single transaction with ETC is about \$45.45.¹ The equation to calculate the financial cost is given below.

¹ 1 ETC = \$45.45 (AUD)- Writing a paper on Apr 20, 2021.

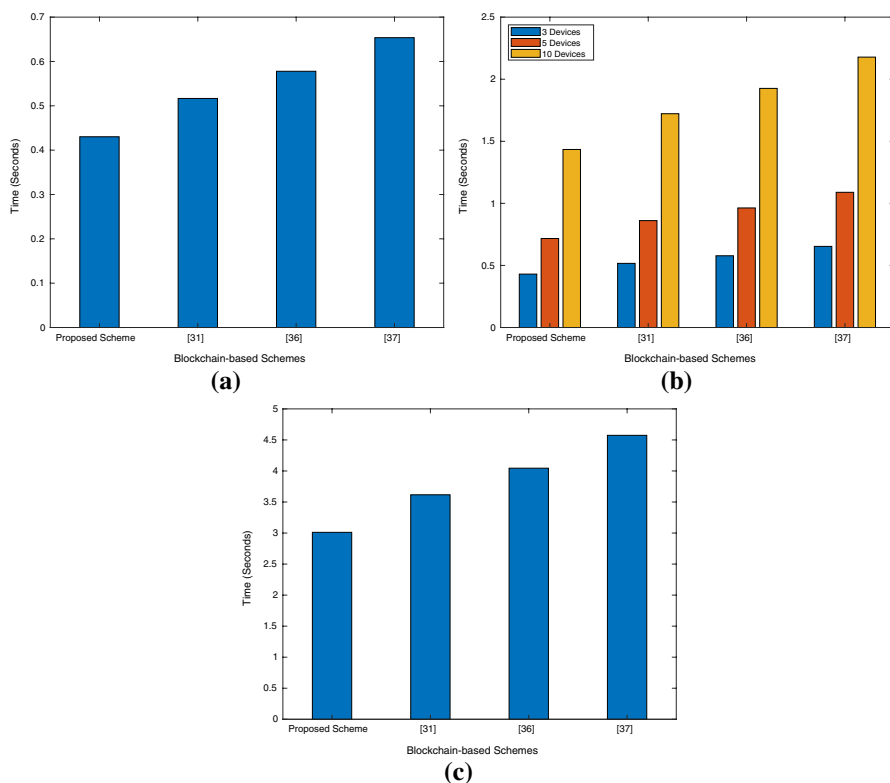


Fig. 14 **a** Computational overhead (miner nodes) **b** computational overhead (IoT devices) **c** overall computational overhead (miner and IoT devices)

$$\begin{aligned}
 \text{Financial Cost} &= \text{No. of Transaction (s)} \times \\
 &\text{Transaction (s) cost in Gas} \times \text{Gas cost in ETC} \times \\
 &\text{ETC cost in AUD}
 \end{aligned} \tag{12}$$

Table 10 describes the estimated total financial cost of deploying the IoT smart home scenario for the number of transactions on the Blockchain. We count the number of transactions required to initialise and associate miner nodes and IoT devices in the Blockchain. Firstly, we calculate the financial cost of all cases (from Case 1 to Case 3) and then the sum of the costs of all cases to present the total cost. The financial cost is calculated in AUD.

We used Gwei² as an ether unit since it is the most commonly used gas price unit in Ethereum Blockchain. We set each transaction approximation cost in Gas as 50000.

² 1 Ether = 0.000000001 Gwei.

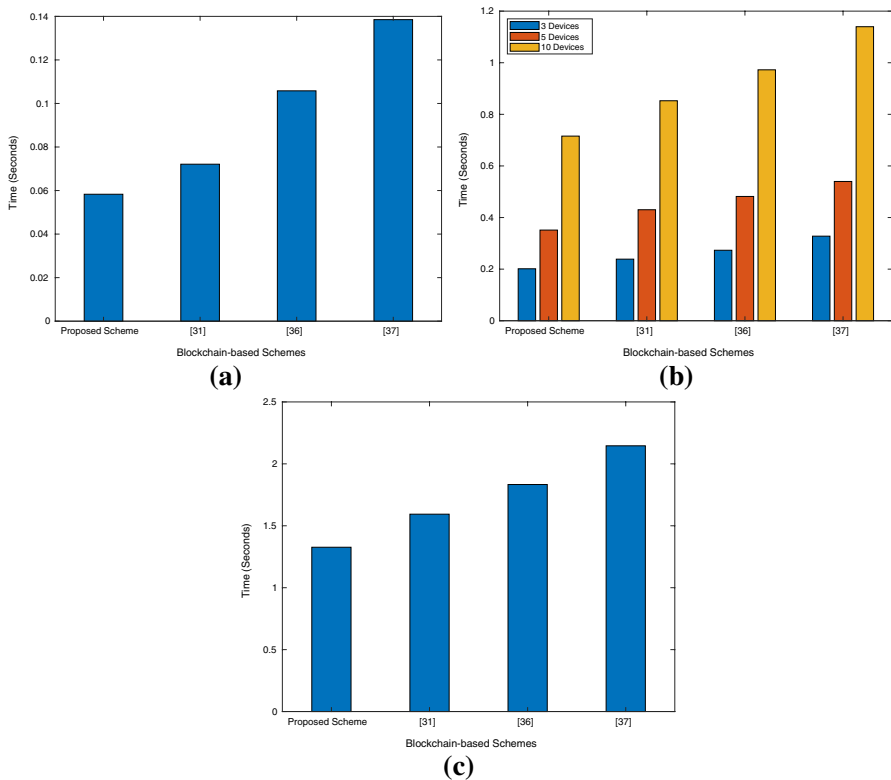


Fig. 15 **a** Communication overhead (miners) **b** communication overhead (IoT devices) **c** overall communication overhead (miner and IoT devices)

7 Conclusion and future work

This paper proposes an efficient and decentralised Blockchain-based authentication scheme for IoT devices that uses smart contracts to ensure secure communication between miner nodes and IoT devices. We designed a secure token mechanism for authenticating IoT devices for secure interaction and communication with other devices using the Ethereum Blockchain. We discussed each component in the architecture of the system and demonstrated the working of the proposed scheme, using the algorithms and sequence diagram. We have set up the Ethereum Blockchain for simulation, in which the experimental results show the significance and performance of the proposed scheme for large IoT networks, while reducing the computational and communication complexity at the initialisation stage. The complete system is modelled, analysed and verified using the HLPN and Z3 solver to demonstrate the correctness of the proposed authentication scheme. In our future work, we plan to investigate IoT network attacks leading to infected authenticated IoT devices and to propose a solution for detecting and mitigating this scenario.

Table 10 Estimated financial cost for an IoT-based smart home

Cases	Miner nodes	IoT devices	Total No. of devices	No. of transactions for initialization and association	Each transaction approx. cost (In Gas)	Total gas consumed	Gas cost (ETC)	Price (AUD)
Case 1	1	3	4	4	50000	400000	0.0004	0.01818
Case 2	1	5	6	6	50000	600000	0.0006	0.02727
Case 3	1	10	11	11	50000	1100000	0.0011	0.049995
Total cost = 0.095445								

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

- Miorandi D, Sicari S, De Pellegrini F, Chlamtac I (2012) Internet of things: vision, applications and research challenges. *Ad hoc Netw.* 10(7):1497–1516
- Gubbi J, Buyya R, Marusic S, Palaniswami M (2013) Internet of things (IOT): a vision, architectural elements, and future directions. *Future Gener Comput. Syst.* 29(7):1645–1660
- Veeramakali T, Siva R, Sivakumar B, Mahesh PS, Krishnaraj N (2021) An intelligent internet of things-based secure healthcare framework using blockchain technology with an optimal deep learning model. *J Supercomput* pp 1–21
- Al-Fuqaha A, Guizani M, Mohammadi M, Aledhari M, Ayyash M (2015) Internet of things: a survey on enabling technologies, protocols, and applications. *IEEE Commun Surv Tutor* 17(4):2347–2376
- Ujjwal K, Garg S, Hilton J, Aryal J, Forbes-Smith N (2019) Cloud computing in natural hazard modeling systems: current research trends and future directions. *Int J Disaster Risk Reduct.* 38:101188
- Khan MA, Abuhasel KA (2020) An evolutionary multi-hidden Markov model for intelligent threat sensing in industrial internet of things. *J Supercomput* pp 1–15
- Nawir M, Amir A, Yaakob N, Lynn OB (2016) Internet of things (IOT): taxonomy of security attacks. In: 2016 3rd International Conference on Electronic Design (ICED), IEEE, pp 321–326
- Kim H, Lee EA (2017) Authentication and authorization for the internet of things. *IT Prof* 19(5):27–33
- Sharma S, Verma VK (2021) Security explorations for routing attacks in low power networks on internet of things. *J Supercomput* 77(5):4778–4812
- Liu J, Xiao Y, Chen CP (2012) Authentication and access control in the internet of things. In: 2012 32nd International Conference on Distributed Computing Systems Workshops, IEEE, pp 588–592
- Mendez Mena D, Papapanagiotou I, Yang B (2018) Internet of things: survey on security. *Inf Secur J A Glob Perspect* 27(3):162–182
- Mabodi K, Yusefi M, Zandiyan S, Irankhah L, Fotuhi R (2020) Multi-level trust-based intelligence schema for securing of internet of things (IOT) against security threats using cryptographic authentication. *J Supercomput* pp 1–26
- Tu YJ, Kapoor G, Piraamuthu S (2021) Security of lightweight mutual authentication protocols. *J Supercomput* 77(5):4565–4581
- Wang P, Chaudhry S, Li L, Li S, Tryfonas T, Li H (2016) The internet of things: a security point of view. *Intern Res*
- Abomhara M et al (2015) Cyber security and the internet of things: vulnerabilities, threats, intruders and attacks. *J Cyber Secur Mobil* 4(1):65–88
- Duan Y, Li J, Srivastava G, Yeh JH (2020) Data storage security for the internet of things. *J Supercomput* pp 1–19
- Lau CH, Alan KHY, Yan F (2018) Blockchain-based authentication in IOT networks. In: 2018 IEEE Conference on Dependable and Secure Computing (DSC), IEEE, pp 1–8
- Nakamoto S (2009) Bitcoin: a peer-to-peer electronic cash system. *Tech. rep.* Manubot
- Swan M (2015) Blockchain thinking: the brain as a decentralized autonomous corporation [commentary]. *IEEE Technol Soc Mag* 34(4):41–52
- Puthal D, Malik N, Mohanty SP, Kougianos E, Yang C (2018) The blockchain as a decentralized security framework [future directions]. *IEEE Consum Electron Mag* 7(2):18–21
- Panarello A, Tapas N, Merlino G, Longo F, Puliafito A (2018) Blockchain and IOT integration: a systematic survey. *Sensors* 18(8):2575
- Liu T, Yuan Y, Yu Z (2021) The service architecture of internet of things terminal connection based on blockchain technology. *J Supercomput* pp 1–21
- Košćál K, Helebrandt P, Belluš M, Ries M, Kotuliak I (2019) Management and monitoring of IOT devices using blockchain. *Sensors* 19(4):856

24. Guin U, Cui P, Skjellum A (2018) Ensuring proof-of-authenticity of IOT edge devices using blockchain technology. 2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber. Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), IEEE, pp 1042–1049
25. Wood G et al (2014) Ethereum: a secure decentralised generalised transaction ledger. Ethereum Proj Yellow Pap 151(2014):1–32
26. Zheng Z, Xie S, Dai H, Chen X, Wang H (2017) An overview of blockchain technology: architecture, consensus, and future trends. In: 2017 IEEE International Congress on Big Data (BigData Congress), IEEE, pp 557–564
27. Luu L, Chu DH, Olickel H, Saxena P, Hobor A (2016) Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, pp 254–269
28. Atlam HF, Wills GB (2019) Intersections between IOT and distributed ledger. Adv Comput 115:73–113
29. Zhu X, Badr Y, Pacheco J, Hariri S (2017) Autonomic identity framework for the internet of things. In: 2017 International Conference on Cloud and Autonomic Computing (ICCAAC), IEEE, pp 69–79
30. Fayad A, Hammi B, Khatoun R (2018) An adaptive authentication and authorization scheme for IOT's gateways: a blockchain based approach. 2018 Third International Conference on Security of Smart Cities. Industrial Control System and Communications (SSIC), IEEE, pp 1–7
31. Hammi MT, Hammi B, Bellot P, Serhrouchni A (2018) Bubbles of trust: a decentralized blockchain-based authentication system for IOT. Comput Secur 78:126–142
32. Leng J, Ruan G, Jiang P, Xu K, Liu Q, Zhou X, Liu C (2020) Blockchain-empowered sustainable manufacturing and product lifecycle management in industry 4.0: a survey. Renew Sustain Energy Rev 132:110112
33. Leng J, Ye S, Zhou M, Zhao JL, Liu Q, Guo W, Cao W, Fu L (2020) Blockchain-secured smart manufacturing in industry 4.0: A survey. IEEE Trans Syst Man Cybern Syst
34. Huh S, Cho S, Kim S (2017) Managing iot devices using blockchain platform. In: 2017 19th International Conference on Advanced Communication Technology (ICACT), IEEE, pp 464–467
35. Durand A, Gremaud P, Pasquier J (2017) Decentralized web of trust and authentication for the internet of things. In: Proceedings of the Seventh International Conference on the Internet of Things, pp 1–2
36. Li D, Peng W, Deng W, Gai F (2018) A blockchain-based authentication and security mechanism for IOT. In: 2018 27th International Conference on Computer Communication and Networks (ICCCN), IEEE, pp 1–6
37. Almadhoun R, Kadadha M, Alhemeiri M, Alshehhi M, Salah K (2018) A user authentication scheme of IOT devices using blockchain-enabled fog nodes. In: 2018 IEEE/ACS 15th International Conference on Computer Systems and Applications (AICCSA), IEEE, pp 1–8
38. Lee JH (2017) Bidaas: Blockchain based ID as a service. IEEE Access 6:2274–2278
39. Lee CH, Kim KH (2018) Implementation of IOT system using block chain with authentication and data protection. In: 2018 International Conference on Information Networking (ICOIN), IEEE, pp 936–940
40. Patwary AAN, Fu A, Battula SK, Naha RK, Garg S, Mahanti A (2020) Fogauthchain: a secure location-based authentication scheme in fog computing environments using blockchain. Comput Commun 162:212–224
41. Shen M, Liu H, Zhu L, Xu K, Yu H, Du X, Guizani M (2020) Blockchain-assisted secure device authentication for cross-domain industrial IOT. IEEE J Sel Areas Commun 38(5):942–954
42. Khalid U, Asim M, Baker T, Hung PC, Tariq MA, Rafferty L (2020) A decentralized lightweight blockchain-based authentication mechanism for IOT systems. Clust Comput pp 1–21
43. Maleh Y, Shojafar M, Alazab M, Romdhani I (2020) Blockchain for cybersecurity and privacy: architectures, challenges, and applications. CRC Press, Boca Raton
44. Leng J, Zhou M, Zhao LJ, Huang Y, Bian Y (2020) Blockchain security: a survey of techniques and research directions. IEEE Trans Serv Comput
45. De Moura L, Bjørner N (2009) Satisfiability modulo theories: an appetizer. In: Brazilian Symposium on Formal Methods, Springer, pp 23–36
46. Jensen K, Rozenberg G (2012) High-level Petri nets: theory and application. Springer Science & Business Media, Berlin
47. Genrich HJ, Lautenbach K (1981) System modelling with high-level petri nets. Theor Comput Sci 13(1):109–135

48. Sibertin-Blanc C (1985) High level petri nets with data structure
49. Jensen K (1983) High-level petri nets. In: Applications and theory of Petri nets, Springer, pp 166–180
50. Vizel Y, Weissenbacher G, Malik S (2015) Boolean satisfiability solvers and their applications in model checking. *Proc IEEE* 103(11):2021–2035
51. Clarke EM, Emerson EA, Sistla AP (1986) Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Trans Program Lang Syst (TOPLAS)* 8(2):244–263
52. Frade MJ, Pinto JS (2011) Verification conditions for source-level imperative programs. *Comput Sci Rev* 5(3):252–277
53. Cimatti A, Griggio A, Schaafsma BJ, Sebastiani R (2013) The mathsat5 SMT solver. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer, pp 93–107
54. Brummayer R, Biere A (2009) Boolector: an efficient SMT solver for bit-vectors and arrays. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer, pp 174–177
55. Jha S, Limaye R, Seshia SA (2009) Beaver: engineering an efficient SMT solver for bit-vector arithmetic. In: International Conference on Computer Aided Verification, Springer, pp 668–674
56. De Moura L, Bjørner N (2008) Z3: An efficient SMT solver. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Springer, pp 337–340

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Khizar Hameed¹  · Saurabh Garg¹ · Muhammad Bilal Amin¹ · Byeong Kang¹

Saurabh Garg
saurabh.garg@utas.edu.au

Muhammad Bilal Amin
bilal.amin@utas.edu.au

Byeong Kang
byeong.kang@utas.edu.au

¹ Syndicate of Technology, Environments and Design, College of Sciences and Engineering, University of Tasmania, Hobart, Australia